

# Summary:

## COSMA: Communication-aware Optimization of Fermionic Simulation Kernels for Modular Quantum Architectures

Enrico Russo, Francesco G. Blanco, Elio Vinciguerra,  
Davide Patti, Giuseppe Ascia, Maurizio Palesi  
*arXiv:2607.09381* (IEEE QCE 2026)

### 1 Main new idea

On modular / multi-core QPUs, fermionic simulation (Trotterized chemistry) is bottlenecked by **inter-core qubit state transfers**, not local gates. Prior compilers typically fix a fermion-to-qubit (F2Q) mapping (JW, BK, ...), synthesize Pauli gadgets, then allocate qubits — treating those stages separately.

**COSMA**’s new idea is joint, communication-aware co-design of three coupled stages:

1. a **genetic search over product-preserving ternary-tree (PPTT) F2Q mappings** (tree shape + mode order), with fitness = downstream transfer cost;
2. **Pauli scheduling** that smooths support changes (Gray-inspired best);
3. a **parity-tree synthesis + multi-core allocator** that builds CNOT trees core-by-core and merges them with a lookahead misplacement score.

On 14 molecular STO-3G benchmarks (14–90 modes, cores of capacity 8), COSMA reports up to  $\sim 2.5\times$  lower communication cost vs. fixed-mapping + Hungarian allocation baselines (median  $\sim 1.7\times$  /  $\sim 60\%$  reduction under Gray scheduling).

### 2 Toy model with concrete numbers

#### Setup

Two cores  $C_0$ – $C_1$  linked by distance  $d = 1$ , capacity  $\kappa = 4$  each.  $N = 6$  qubits. One Trotter step has  $T = 4$  Pauli gadgets with supports

$$S_1 = \{1, 2, 3\}, \quad S_2 = \{1, 2, 4\}, \quad S_3 = \{4, 5, 6\}, \quad S_4 = \{3, 5, 6\}.$$

Communication cost (paper Eq. (14)) counts total core-distance moved by every qubit between consecutive slices.

#### Naive pipeline (JW + magnitude + fixed CNOT chain + Hungarian)

JW mapping spreads modes so supports straddle cores often. Magnitude ordering ignores support smoothness: support-delta surrogate (Eq. (19))

$$\Delta_{\text{sched}} = \sum_{t=2}^T |S_{t-1} \triangle S_t| = |\{3\} \triangle \{4\}| + |\{1, 2\} \triangle \{5, 6\}| + |\{4\} \triangle \{3\}| = 2 + 4 + 2 = 8.$$

Each gadget needs a parity tree merged across cores  $\Rightarrow$  roughly 2 transfers per gadget  $\times$  forward+reverse  $\approx$  16 unit-distance transfers. **Naive cost**  $\approx$  16.

### COSMA-style choices

- Mapping places modes so  $S_1, S_2$  live mostly on  $C_0$  and  $S_3, S_4$  on  $C_1$ .
- Gray-inspired order:  $(S_1, S_2, S_4, S_3)$  so consecutive supports change by only 1–2 qubits:  $\Delta_{\text{sched}} = 2 + 2 + 2 = 6$  (25% smoother).
- Allocator builds a local parity chain inside each core, then merges at one meeting core; lookahead keeps well-placed qubits still.

Suppose transfers drop to 6 unit moves. **COSMA cost**  $\approx$  6 — about **2.7** $\times$  cheaper, in the same ballpark as the paper’s peak  $\sim 2.5\times$  / median  $\sim 1.7\times$ .

| Pipeline                | $\Delta_{\text{sched}}$ | Transfers | Cost ratio                       |
|-------------------------|-------------------------|-----------|----------------------------------|
| JW+MAG+HQA (naive)      | 8                       | 16        | 1.00                             |
| COSMA map+GRAY+co-alloc | 6                       | 6         | <b>0.38</b> ( $\sim 2.6\times$ ) |

## 3 Paper math walkthrough

### Simulation kernel

Electronic Hamiltonian  $\rightarrow$  Majorana form  $\rightarrow$  Pauli sum

$$H = \sum_j \alpha_j P_j, \quad e^{-iHt} \approx \left( \prod_j e^{-i\alpha_j P_j t/r} \right)^r. \quad (1)$$

Each factor  $e^{-i\theta P}$  is a **Pauli gadget**: basis change  $\rightarrow$  CNOT parity tree  $\rightarrow R_Z(2\theta)$  on the root  $\rightarrow$  reverse tree.

### Objective (what COSMA minimizes)

With layout  $\ell_l(q)$  = core of qubit  $q$  in slice  $l$ ,

$$\text{cost}(\mathcal{M}, \pi, \{\mathcal{T}_t\}, \{\ell_l\}) = \sum_{l=1}^{L-1} \sum_{q=1}^N d(\ell_l(q), \ell_{l+1}(q)), \quad (2)$$

and

$$\min_{\mathcal{M}, \pi, \{\mathcal{T}_t\}, \{\ell_l\}} \text{cost}(\mathcal{M}, \pi, \{\mathcal{T}_t\}, \{\ell_l\}). \quad (3)$$

Symbols:  $\mathcal{M}$  PPTT F2Q mapping;  $\pi$  Pauli order;  $\mathcal{T}_t$  parity tree for term  $t$ ;  $d$  shortest-path core distance.

### Why mapping matters

PPTT search space size is  $A_N^{(3)} \cdot N!$  with ternary Fuss–Catalan  $A_N^{(3)} = \frac{1}{2N+1} \binom{3N}{N}$ . Different trees change Pauli supports  $S_t$ , hence which gadgets cross cores.

## Gray scheduling surrogate

With support mask  $M_t$ ,

$$g(P) = M(P) \oplus (M(P) \gg 1), \quad \Delta_{\text{sched}} = \sum_{t=2}^T d_H(M_{t-1}, M_t). \quad (4)$$

Sorting by  $g(P)$  clusters similar supports — fewer qubits to reshuffle.

## Allocator lookahead

Misplacement of qubit  $q$  at term  $t$  (window  $W$ , discount  $\gamma$ ):

$$\bar{d}_t(q) = \sum_{\tau \in W_t(q)} \gamma^{\tau-t} \delta_\tau(q), \quad (5)$$

where  $\delta_\tau(q)$  is average core-distance from  $q$  to co-support partners. Meeting core for term  $t$ :

$$c_t^* = \arg \min_{c \in A(S_t)} \sum_{q \in S_t} \frac{d(c, \ell_t(q))}{1 + \bar{d}_t(q)}. \quad (6)$$

Forward merge + backward co-location use these scores to decide which qubit moves when parity subtrees on different cores must be joined — mapping toy transfers onto Eq. (2).

## 4 Figures from the paper

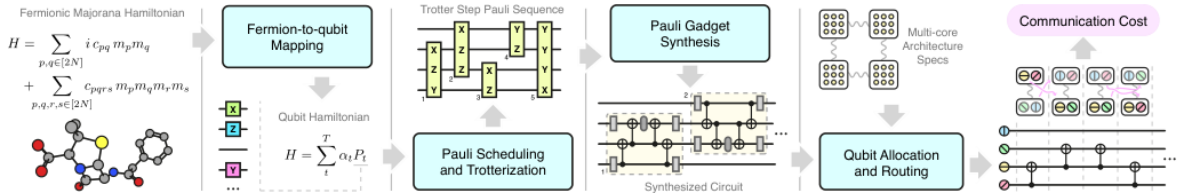


Fig. 1: Overview of the compilation flow for quantum simulation kernels on modular architectures.

## II. BACKGROUND

Fig. 1 summarises the steps required to evaluate the com-

The *support* of  $P_j$  is the set of qubits on which it acts non-trivially,

$$S_i = \text{SUDD}(P_i) := \{k \mid \sigma_{z,k} \neq I\}.$$

**Figure 1: FIG. 1 (paper) — compilation flow.** Molecule  $\rightarrow$  F2Q mapping  $\rightarrow$  Pauli Hamiltonian  $\rightarrow$  scheduled Trotter sequence  $\rightarrow$  Pauli-gadget synthesis  $\rightarrow$  qubit allocation/routing on a multi-core grid  $\rightarrow$  **communication cost** (inter-core state transfers).

*How to read Fig. 1.* Every arrow is a compiler stage COSMA co-optimizes. The rightmost “Communication Cost” panel is the fitness signal fed back into the genetic F2Q search.

*How to read Fig. 6.* Cost Eq. (2) literally counts those red arcs (weighted by link distance). COSMA’s job is to choose mapping, order, and trees so fewer arcs appear.

*How to read Fig. 7.* The loop “Population  $\rightarrow$  Fitness (communication cost)  $\rightarrow$  Survival” is the novelty: mapping is scored by modular communication, not Pauli weight alone.

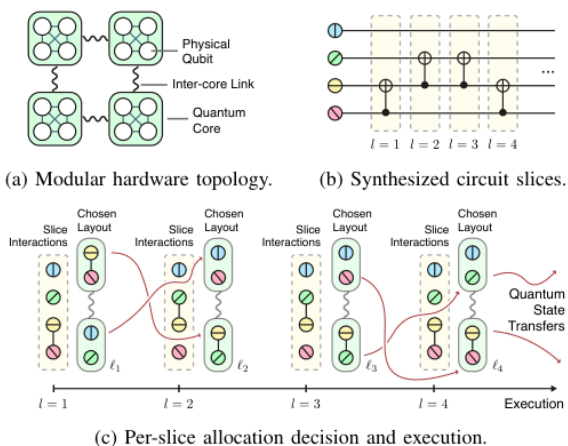


Fig. 6: Modular quantum-architecture model and allocation process for a sliced circuit.

Fig. 3 shows three possible orderings of Pauli terms in the multi-Hamiltonian. Distributing the qubits in their respective

**Figure 2: FIG. 6 (paper) — modular model + sliced allocation.** (a) Cores with local qubits and inter-core links. (b) Circuit sliced into parallel CNOT groups. (c) Per-slice layouts; red arcs = quantum state transfers when a qubit’s core changes.

*How to read Figs. 8–9.* Shorter blue bars = fewer state transfers. Advantage holds from 14-mode water to  $\sim 90$ -mode cytosine. Abstract’s “up to  $2.5\times$  / median  $1.7\times$ ” is this gap rephrased as a speedup factor.

*How to read Figs. 10–12.* Most of the big win in Figs. 8–9 is joint co-optimization (allocator+mapping together), not mapping alone (only  $+11\%$ ). Scheduling choice is large when the allocator is fixed: smooth supports (GRAY) matter. Fig. 12 says the GA is practical with GPU for the inner loop.

## 5 Takeaway

COSMA’s novelty is treating **F2Q mapping, Pauli order, and parity-tree/allocation as one communication objective** for modular chemistry kernels. The concrete win is fewer inter-core teleports/transfers (up to  $\sim 2.5\times$ ), driven especially by Gray-smoothed schedules plus a lookahead parity-forest allocator — with a genetic PPTT mapper using that cost as fitness.

The objective is to minimise the total inter-core communication cost incurred when qubits migrate between cores across consecutive slices. Let  $c_{q,l}$  denote the core assigned to logical qubit  $q$  in slice  $l$ , let  $Q$  be the total number of logical qubits,  $L$  the total number of slices, and let  $\mathbf{D}$  be the inter-core distance matrix. According to this model [31], [35], the total transfer cost is

$$\sum_{l=1}^{L-1} \sum_{q=1}^Q \mathbf{D}[c_{q,l}, c_{q,l+1}]. \quad (13)$$

An assignment of logical qubits to cores is also referred to as a *layout*, and we denote the layout of slice  $l$  by  $\ell_l$ . The sequence of layouts across slices defines the qubit-routing process, as it determines how logical qubits move across cores during circuit execution.

Fig. 6c shows a simple case in which the four slices of Fig. 6b are allocated on a two-core architecture. During execution, changes in layout correspond to inter-core qubit transfers.

### III. PROBLEM FORMULATION

We formalise the optimization problem addressed by COSMA, and analyse the complexity of its assignment sub-

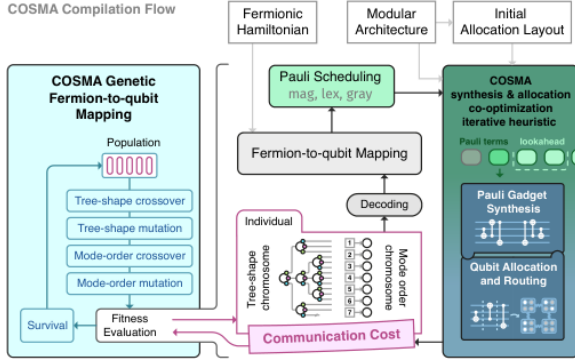


Fig. 7: Overview of the COSMA framework.

parity trees and the qubit-interaction structure seen by the allocator. The ordering  $\pi$  controls the smoothness of the support sequence across consecutive gadgets, potentially reducing

Starting from a Majorana Hamiltonian (9), COSMA selects a fermion-to-qubit transformation, expands and aggregates the resulting Pauli terms, imposes an execution order on them, and finally performs a topology-aware allocation that simultaneously synthesises the CNOT parity tree for each Pauli gadget and manages qubit placement across cores. Let  $(P_1, \dots, P_T)$  denote the ordered sequence of Pauli terms,  $S_t = \text{supp}(P_t)$  its support, and  $M_t \in \{0, 1\}^n$  its support mask. The three stages are coupled: the mapping determines the possible supports; the scheduling determines the temporal order in which the sets  $S_t$  are encountered; and the allocation stage both synthesises the parity circuit for each term and determines the inter-core transfers needed to make every CNOT executable.

#### A. Fermion-to-Qubit Mapping

We consider both standard F2Q mappings and optimised tree-based mappings as baselines. The standard baselines are JW, BK, PE, and JKMN mappings. Beyond these baselines, we optimise the ternary tree shape and the assignment of fermionic modes to tree nodes using a genetic algorithm [36].

**Figure 3: FIG. 7 (paper) — COSMA framework.** Genetic individuals = (tree-shape chromosome, mode-order chromosome). Fitness = full schedule+allocate cost. Downstream: mag/lex/gray scheduling and iterative synthesis/allocation with lookahead.

where

$$F_t(q, c) = \sum_{\substack{\tau \in W_t(q) \\ \tau > t}} \gamma^{\tau-t} \sum_{\substack{r \in S_\tau \\ r \neq q}} d(c, \ell_t(r)) \quad (27)$$

is the discounted future interaction distance of placing qubit  $q$  on core  $c$ . The qubit with the larger gain is moved; if the core is full the same swap strategy of forward pass is applied. Each co-location move generates a new circuit slice, updates the layout for subsequent edges in the same traversal and increases the running communication cost by  $d(c_p, c_q)$ .

After both phases complete, the allocator has produced a fully executable parity CNOT tree for  $P_t$  together with the complete sequence of inter-core transfers required to realise it in both forward and reverse passes. The process repeats for  $t = 1, \dots, T$ , adding circuit slices as needed and choosing qubit layout for each of them.

## V. EVALUATION

We evaluate COSMA on 14 molecular workloads from the PubChem database [39] in the STO-3G basis: water, ammonia, methane, carbon dioxide, urea, glycine, alanine, ethylene, benzene, ethanol, acetone, acetic acid, uracil, and cytosine. The molecular integrals and electronic Hamiltonians are generated with PySCF [40], after which we convert them into the sparse Majorana representation used by our pipeline, and validate the resulting Hamiltonians against OpenFermion [41]. These workloads span 14 to 90 fermionic modes and approximately  $4.2 \times 10^3$  to  $6.5 \times 10^6$  sparse Majorana terms. For each workload, we consider a 2D grid multi-core architecture with fixed core capacity 8 and choose the smallest rectangular grid with sufficient total capacity, ranging from  $(1 \times 2, 8)$  for the smallest molecules up to  $(3 \times 4, 8)$  for the largest ones.

For the genetic mapping stage we consider population size 50 and 25 generations, with transfer cost as the optimization objective. Communication is always reported as the total

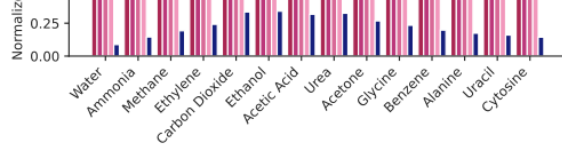


Fig. 9: Comparison against baselines considering Gray-like Pauli scheduling.

weighted inter-core transfer cost defined by the target architecture as in [31], [35], [42].

We consider three experiment families. First, we compare our full pipeline against baselines. Second, we isolate the effect of the fermion-to-qubit mapping while holding scheduling and allocation fixed. Third, we isolate the effect of scheduling while holding the mapping and allocation fixed. In all plots, molecules are ordered by increasing number of fermionic modes.

### A. Comparison Against Baselines

The comparison baselines use fixed fermion-to-qubit mappings (JW, BK, PE, and JKMN) combined with two scheduling strategies, namely Gray-inspired and magnitude ordering. For allocation we consider the Hungarian qubit assignment algorithm (HQA) [35]. To make the Hungarian allocator applicable, each Pauli gadget is first synthesized into an index-ordered CNOT chain, where the support qubits are connected from lower index to higher index. This produces an explicit sequence of two-qubit gates, which is then sliced and passed to Hungarian assignment on the same grid architecture. We compare this baseline family against our full COSMA pipeline, which combines the genetic tree-based mapping, same scheduling, and the synthesis and allocation co-optimizer.

**Figure 4: FIGS. 8–9 (paper) — full pipeline vs. baselines.** Normalized inter-core transfer cost across 14 molecules (Water→Cytosine). Pink/red bars: fixed JW/BK/PE/JKMN + HQA. Dark blue: full COSMA. Fig. 8 uses magnitude scheduling; Fig. 9 uses Gray-inspired (median **59.7%** reduction).

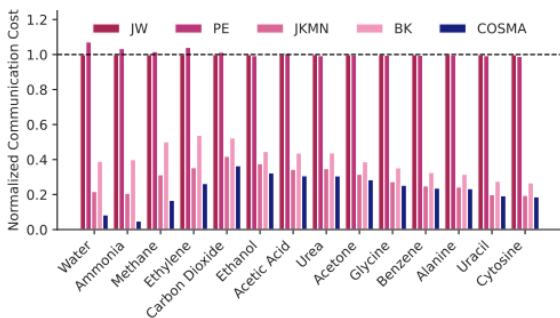


Fig. 10: Impact of mapping on communication cost.

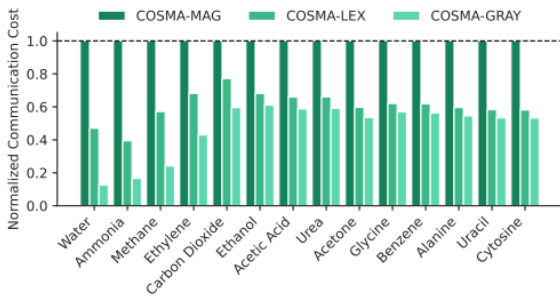


Fig. 11: Impact of scheduling on communication cost.

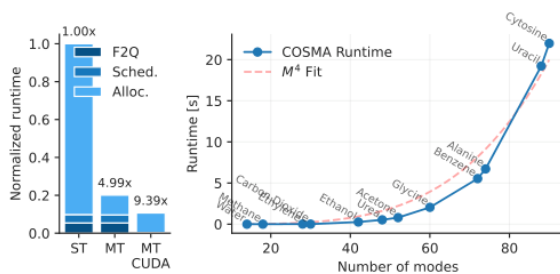


Fig. 12: Single-thread (ST), multi-thread (MT) and CUDA accelerated implementation runtime comparison for Ethylene (left). COSMA runtime for different numbers of modes (right).

generations and population size 50 for all molecules. As a result, the optimizer likely explores a smaller fraction of the candidate mappings for the larger instances, which can reduce the gains attainable from mapping optimization alone.

*b) Impact of scheduling:* To isolate scheduling, we fix the genetic mapping and the parity-tree-aware allocator, and compare Gray-inspired, lexicographic, and magnitude ordering. As shown in Fig. 11, Gray-inspired scheduling is consistently the strongest of the three: relative to lexicographic ordering, the median transfer-cost ratio is 0.89 (median reduction 10.5%), while relative to magnitude ordering the median ratio is 0.54 (median reduction 46.0%). This confirms that ex-

**Figure 5: FIGS. 10–12 (paper) — ablations + runtime.** **Fig. 10:** mapping-only ablation (same Gray scheduler + COSMA allocator): genetic PPTT mapping median **11.4%** better than best fixed mapping. **Fig. 11:** scheduling-only ablation: GRAY beats LEX by  $\sim 10.5\%$  and MAG by  $\sim 46\%$  (median). **Fig. 12:** runtime — CUDA  $\sim 9.4\times$  vs. single-thread; scales  $\sim M^4$  with modes.